CS 101
Computer Programming and Utilization

Instructor: Rohit Gurjar

All material based on Prof. Abhiram Ranade's book and slides
(and previous course instructors)

This lecture

- Introduction to the topic
- Administrative details
- A simple program

Computer/Computing

- Everywhere: smartphones, laptops, TV, video games, aeroplanes, drones, …
- Computing:
  - When you book a train ticket
  - Search on internet
  - Find the shortest route on maps
  - Instagram filter
  - Sending a message on chat
  - Creating a meme
- Goal of the course: How to make computer do such things

What is a computer?

- A computer is a giant electrical circuit that can be instructed/programmed to do various tasks
  - Receive data from the external world
  - data = numbers, images, sounds, can be represented using numbers and fed to a computer, maps, data from sensors,
  - Process the data it receives
  - Send the results back to the external world
- What process/computing is performed?
  - Determined by a program loaded in the computer

Program

- Program: a precise description of the computation we want the computer to perform
- By feeding different programs to a computer you can make it do different computations.
- This course tells you how to construct (“write”) programs.
- Programming language: a language (grammar, vocabulary) to give instructions computers.
- Why not natural languages (English, Marathi)?
  Because ambiguous, not standardised, not expressive enough.
- Exercise: Try describing the procedure of dividing a number by another in English/

Programming for engineers/scientists?

- For example, you will be writing programs
- CSE: to make computers more energy efficient, to verify if softwares/apps developed by others work as desired, to make computers more intelligent.
- EC: to analyze financial data, to build economic models, to simulate economies and forecast
- MM: simulations to predict how materials evolve under certain conditions, predict material properties based on composition

Programming for engineers/scientists?

- For example, you will be writing programs
- CL: to build and simulate mathematical models of reactors, entire plants, to build automated control systems for Chemical plants
- EE: to create and test circuit designs virtually, signal processing
- MA: to solve differential equations, integrals via numerical methods, to visualize mathematical concepts, testing conjectures

How do you get good at programming?

- By doing it yourself
- Almost all learning will happen in the lab.
- TAs will help if you are stuck. You can also discuss ideas with your friends. But, don't directly ask for solutions.
- Write the program yourself.
- Initially, it may be confusing and frustrating. Be patient and continue practicing.
- Prohibited: finding solutions on internet, ChatGPT, Claude, Gemini, Perplexity etc.

Administrative details

Schedule

- Lectures:
  - D2 : LA 201 (slot 11) Tue, Fri 3:30 - 4:55 PM
  - D4 : LA 202 (slot 9) Mon, Thu 3:30 - 4:55 PM
- Weekly lab: (SL1/SL2)
  - P10, P20 : Mon 9:30-11:30 AM
  - P8, P19 : Tue 9:30-11:30 AM
  - P9, P22 : Thu 9:30-11:30 AM
  - P7, P21 : Fri 9:30-11:30 AM
- Extra help session: Sat 2:00-4:00 PM.
- Holidays: Make up lab for 15 Aug, 5 Sep (Fri) on next day (Saturday 2-4 pm)
- Quizzes (written): Aug 20, Oct 15 (Wed) 8:30-9:10 am.
- Lab exams: 8 Sep-12 Sep (respective Lab slots), 3 Nov - 7 Nov (respective lab slots)
- Midsem, Endsem Exams: will be scheduled by TTC.

People

- Instructor: Rohit Gurjar
  Email: rgurjar@cse.iitb.ac.in
- Course Manager: Firuza Aibara (on leave till mid sem)
  Email: firuza@cse.iitb.ac.in
- Managing TA: Sarfaraz Equbal
  Email: sequbal@cse.iitb.ac.in
- 5 RAs (cs101-ras@cse.iitb.ac.in) managing the lab sessions.
- A group of 10-12 students will have a TA.
- Each of the 8 lab sessions will have a Head TAs.
  - Will be shared next week.

Communication

- Bodhi Tree (robin.bodhi.cse.iitb.ac.in): weekly lab submission, lab exams, announcements, marks, discussion forum.
- Course Webpage (https://www.cse.iitb.ac.in/~cs101/): All lecture slides and C++ programs.
- For any doubts from lectures/discussions, use BodhiTree discussion forum, or reach instructor after the class, or email.
- For any issues related to lab, marks in exams etc, Email/talk to head TA.
- For any other issues, Email/talk to instructor and managing TA.

Attendance

- Lectures:
  - As per institute rules, minimum 80% attendance required.
  - If attendance is below 80%, you get DX grade and you have repeat the course.
  - A TA will take attendance in every class from second week onwards.
- Weekly lab:
  - Each lab (from second week onwards) will have some marks which will count towards your grade.
  - Marks are for attending and making a genuine attempt at the problem, getting the correct solution or something close it.
  - If absent for medical or other emergency, email to Head TA with documentation

Evaluation

- Quizzes : 5 + 5 %
- Lab exams: 15 + 15 %
- Midsem: 20 %
- Endsem: 30 %
- Weekly labs: 10 %
- For any quizzes/exams missed due to medical reasons, there will be a make-up exam (theory+lab mix) after the endsem.

Academic Honesty

- Do not copy code from any source (classmates, Internet, ChatGPT, ...); do not send code to each other.
- Every line of submitted code must be written by yourself.
- You can consult references for syntax, formats, etc.
- You can discuss code, concepts with friends.
- When in doubt about taking some action, ask TAs or instructor.
- Do not copy in the tests and exams.
- Suspected academic malpractice will be reported to D-ADAC.

Resources

- Textbook: “An introduction to programming through C++”, Abhiram Ranade, McGraw Hill Education, 2014.
  - Integrated with use of simplecpp
  - will be shared on BodhiTree
- Abhiram Ranade’s NPTEL course
  - https://www.youtube.com/playlist?list=PLEAYkSg4uSQ2qzihjdDEseWrrY1DyxH9P

Give us Feedback

- Initial survey (on CS101 homepage)
  - Please let us know if you need a scribe or any help with other issues.
  - Language issues.
- Link for continuous anonymous feedback (on CS101 homepage)
- TALK to me, TAs.
- We want you to enjoy this course, learn from it, apply your learning to future courses and projects.

Getting started with programming

The C++ Programming Language

- Designed by Bjarne Stroustrup, 1980s.
- Evolved out of the C programming language.
- C++ is a powerful, complex language.
- We will not study all of it.
- We will lay the foundation for learning advanced features later.

The Programming Environment

- Initial weeks: C++ augmented with Simplecpp
- Simplecpp is a C++ library developed in IITB by Prof. Abhiram Ranade
  - Provides facilities convenient to learners
    1. Simplified syntax
    2. Graphics programming – more fun!
  - Download from www.cse.iitb.ac.in/~ranade/simplecpp
    Available as Linux/Mac OS library or as IDE for windows and Linux
- Later weeks: Only C++
  - We may continue to use Simplecpp graphics

Let us write some simple C++ programs

- The programs will draw pictures on the screen.
- Use "Turtle Simulator" contained in simplecpp
  - Based on Logo: A language invented for teaching programming to children by Seymour Pappert et al.
  - We "drive" a "turtle" on the screen!
  - To drive the turtle you write a C++ program.
  - Turtle has a pen, so it draws as it moves.

Drawing pictures seems too much fun?

"You master picture drawing, you master programming!"

The first program

```
#include <simplecpp>
main_program{
  turtleSim();
  forward(100);  right(90);
  forward(100);  right(90);
  forward(100);  right(90);
  forward(100);
  wait(5);
}
```

- "Use simplecpp facilities"
- Main program begins
- Start turtle simulator
  - Creates window + turtle at center, facing right
- `forward(n)`:
  - Move the turtle `n` pixels in the direction it is currently facing.
- `right(d)`:
  - Make turtle turn `d` degrees to the right.
- `wait(t)` :
  - Do nothing for `t` seconds.

How to run the program

- First install simplecpp on your computer. TAs will help with this in the first week.

1. Type the program in a file, using a text editor.
   - For example, vim (from terminal), gedit (on linux), TextEdit (on mac)
2. Open Terminal, go to the folder where you have created the file
   (can use `cd` command, or open the folder in terminal using right-click).
3. Compile the program by typing the following on Terminal, and pressing enter / return

```
s++ square.cpp
```

4. Execute / run the program by typing the following on Terminal, and pressing enter / return

```
./a.out
```

Exercises

- Write a program to draw
  - A smaller square
  - A rectangle
  - An equilateral triangle
  - A pentagon
  - Remember that the external angles of a polygon add up to 360 degrees.

Repetition in code — use `repeat`

```
#include <simplecpp>
main_program{
  turtleSim();
  forward(100);  right(90);
  forward(100);  right(90);
  forward(100);  right(90);
  forward(100);
  wait(5);
}
```

```
#include <simplecpp>
main_program{
  turtleSim();
  repeat(4){
      forward(10);
      right(90);
  }
  wait(5);
}
```

- Both the programs draw a square.
- But, the final direction of the turtle is different.

Use of repeat statement

```
repeat(n){
    ⋮
  body
    ⋮
}
```

`n` must be a positive number

`body` is one or more statements

- What will it do?
  - `body` is executed `n` times.
- This is a “loop”. There are other kinds of loop, which we will see later.
- Each execution of the `body` is called an “iteration”.

How to draw a polygon

```
#include <simplecpp>
main_program{
  turtleSim();
  cout << “How many sides?”;
  int nsides;
  cin >> nsides;
  repeat(nsides){
    forward(100);
    right(360.0/nsides);
  }
  wait(10);
}
```

`cout << msg;`

- Print message `msg` on the screen.

`int nsides;`

- Reserve a cell in memory in which I will store some integer value, and call that cell `nsides`.
- You choose the name as you wish, almost!

- `int` : abbreviation of “integer”

`cin >> nsides;`

- Read a value from the keyboard and put it in the cell `nsides`.

`repeat(nsides)`

- repeat as many times as content of `nsides`

`360.0/nsides`

- result of dividing 360 by content of `nsides`.

More simplecpp commands

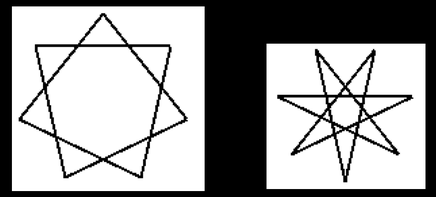
- `left(A)` : turn left A degrees. Equivalent to `right(-A)`
- `penUp()`, `penDown()`: Causes the pen to be raised, lowered
  - Drawing happens only if the turtle moves while the pen is low.
- `hide()` : hide the turtle (triangle).
- `sqrt(x)` : square root of x.
- `sine(x)`, `cosine(x)`, `tangent(x)` : x should be in degrees.
- `sin(x)`, `cos(x)`, `tan(x)` : x should be in radians.
- Also commands for `arcsine`, `arccosine`, `arctangent`... See book.

What we have learnt so far

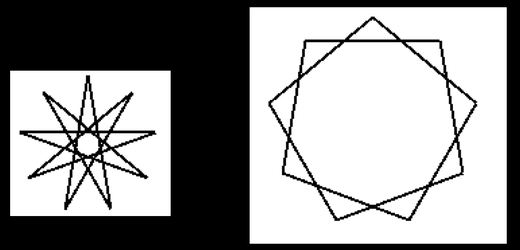
- `Repeat()` causes repeated executions of some statements.
- `cin` and `cout` statements can be used to read from keyboard and to type messages to the screen.
- We have commands to compute mathematical functions as well as lift the pen up and down.

Exercise: Stars

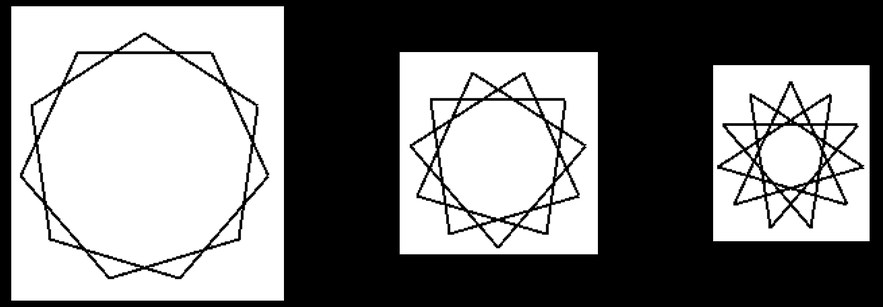
5-star



7-stars

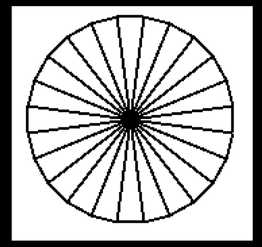


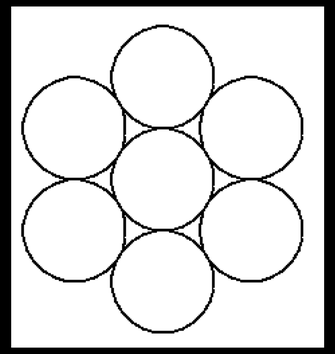
9-stars

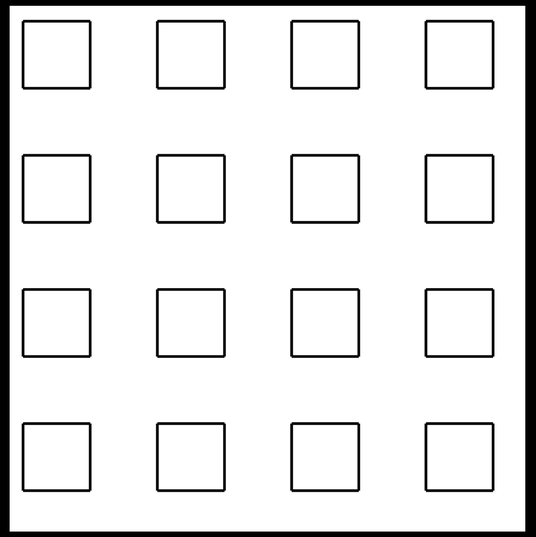


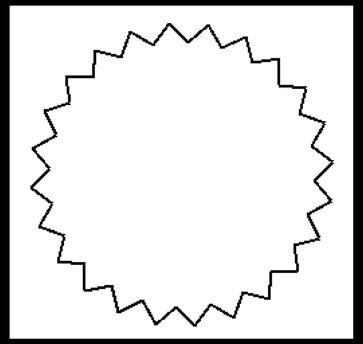
11-stars

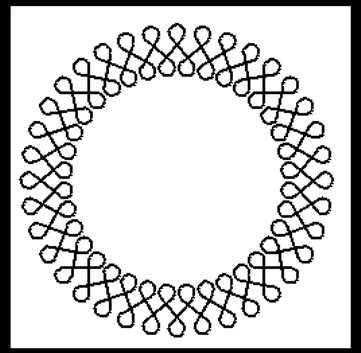
Exercises











Language Syntax

- Syntax = grammatical rules indicating how commands must be written.
- Syntax of programming languages is very strict, e.g.
  - “`right(90);`” cannot be written as “`right 90;`”.
  - “`penUp()`” cannot be written as “`penup()`” or “`penUp`”, i.e. without parentheses.
  - We will later learn other kinds of statements which will have their own syntax which must be adhered to.
- Lot of flexibility is still allowed, e.g.
  - Wherever a number is acceptable, often an ”expression” such as `360/n` is acceptable
  - repeat statement is allowed wherever other statements are allowed, e.g. we can have a repeat inside another repeat.

Comments

- A program will be executed on a computer, but it will also be read by people.
- Sometimes readers may not understand why the program is written the way it is written.
- To help such human readers, you can place “comments” in your program.
  - Anything placed between /* and */ is a comment
  - Anything between // and end of line is a comment
  - A comment is meant only for human readers and is ignored by the computer during execution.

A program with Comments

```
/* Author: Abhiram Ranade
Program to draw polygon */
#include <simplecpp>
main_program{
  turtleSim();
  cout << “How many sides?”;
  int nsides; cin >> nsides;
  repeat(nsides){
    forward(100);
    right(360.0/nsides);  // Exterior angle of an n sided polygon is 360/n
  }
  wait(10);
}
```

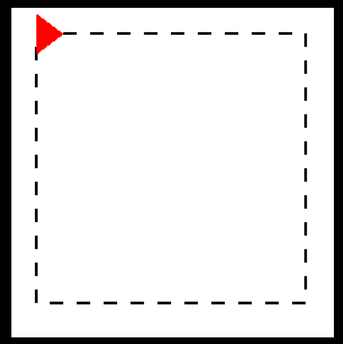
Indentation

```
#include <simplecpp>
main_program{
    turtleSim();
    cout << “How many sides?”;
    int nsides;
    cin >> nSides;
    repeat(nSides){
        forward(100);
        right(360.0/nSides);
    }
    wait(10);
}
```

- You will notice that at the beginning of some lines some space is inserted.
- This space is called indentation.
- The indentation allows you to quickly see which statements constitute the body of the repeat, which statements are part of the main program.
- The general rule: if X is ”inside” Y, then put four additional spaces before every line of X.
- Also note how the { and } are placed.
- Indentation is very helpful for human readers.
- Indentation is ignored during execution.

Repeat within repeat (nested repeat)

```
repeat(4){
    repeat(10){
        forward(10); penUp();
        forward(10); penDown();
    }
    right(90);
}
```



The above program makes this dashed square.

How nested repeat executes

```
repeat(2){
      xxx
     repeat(3){
          yyy
     }
     zzz
}
```

The code on left will execute the statements in the following manner.

```
xxx
yyy
yyy
yyy
zzz
xxx
yyy
yyy
yyy
zzz
```

How nested repeat executes

What will the following program print?

```
#include <simplecpp>
main_program{
  cout << “a”;
  repeat(5){
    cout << “b”;
    repeat(2){  cout << “c”; }
    cout << “d”;
  }
}
```

Commonly used terminology

- “Control is at statement w”: Computer is currently executing statement w.
- “Control flow”: The order in which statements get executed.
  - Execution starts at top and goes down.
  - Retraced if there is a repeat statement.
- Variable: region of memory designated for storing some value you need
- Example: `nsides` which we saw earlier.
- Named so because the value stored in the region can vary
- How to change the value: later.

Why picture drawing

- The calculations/actions needed to solve any problem will contain patterns
  - For example, a sequence of calculations may be repeated
- Key programming activity: The pattern in the calculations must be mirrored by appropriate statements in program.
  - If some calculation is to be repeated 5 times, use repeat(5){} rather than writing out the statement 5 times
- Interesting pictures also contain patterns.
- To draw interesting pictures you need to use repeat statements competently
- By drawing interesting pictures you get some practice at identifying patterns and implementing them in your programs.
- Similar ideas may be used in programming a drone, 3D printer, animation.

Spirit of the course

- Learn C++ statements / concepts.
- Understand patterns in the calculations that you want to do
  - Very important in all programming, not just drawing.
- Goal: if you can solve a problem by hand, possibly taking an enormous amount of time, by the end of the course, you should be able to write a program for it.
- Learn new ways of solving problems!
- Do not be afraid of using the computer.
- “What if I write xyz in my program instead of pqr?” : Just do so and find out.
  - Be adventurous.
- Exercise your knowledge by writing programs – that is the real test.

Data representation and computation

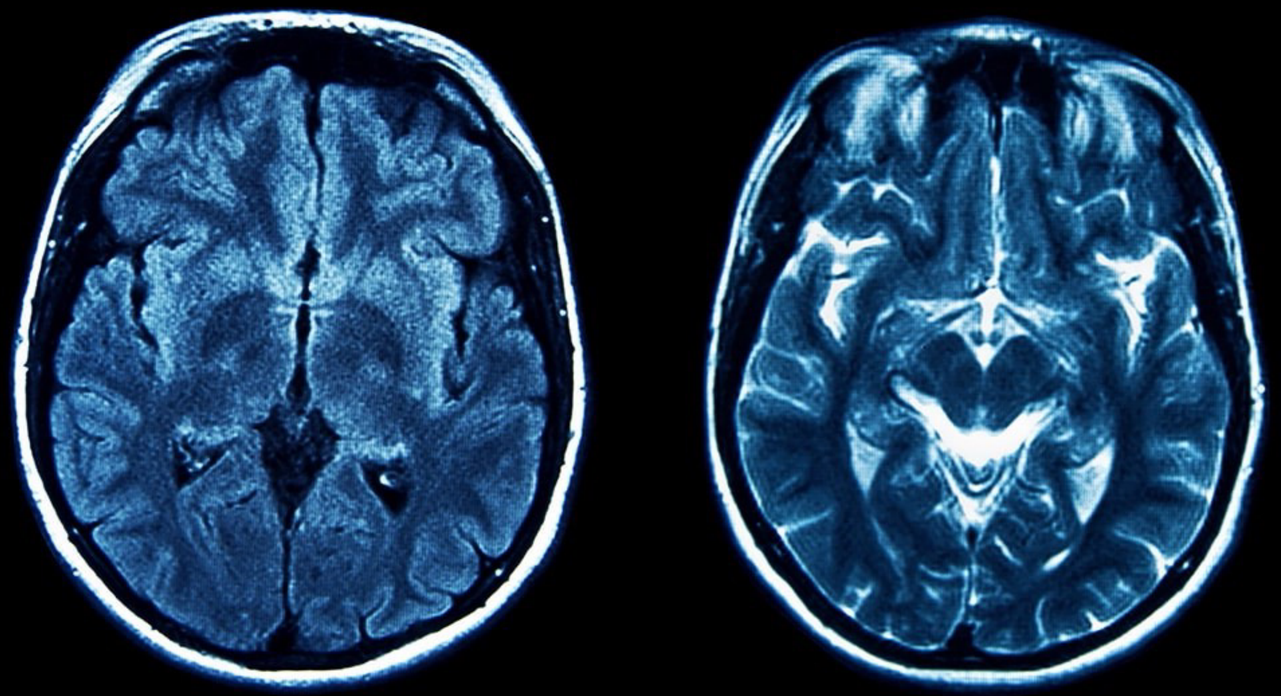
A basic question

- How is a computer able to do so many things?
  - Search for information
  - Process pictures, identify features, edit them
  - Create animation
  - Play chess
- Most real life problems can be viewed as mathematical problems on numbers
- A computer is good at solving math problems

What is in this picture?

https://en.wikipedia.org/wiki/File:Jackson%27s_Chameleon_2_edit1.jpg

MRI of a brain: does it have a tumor?

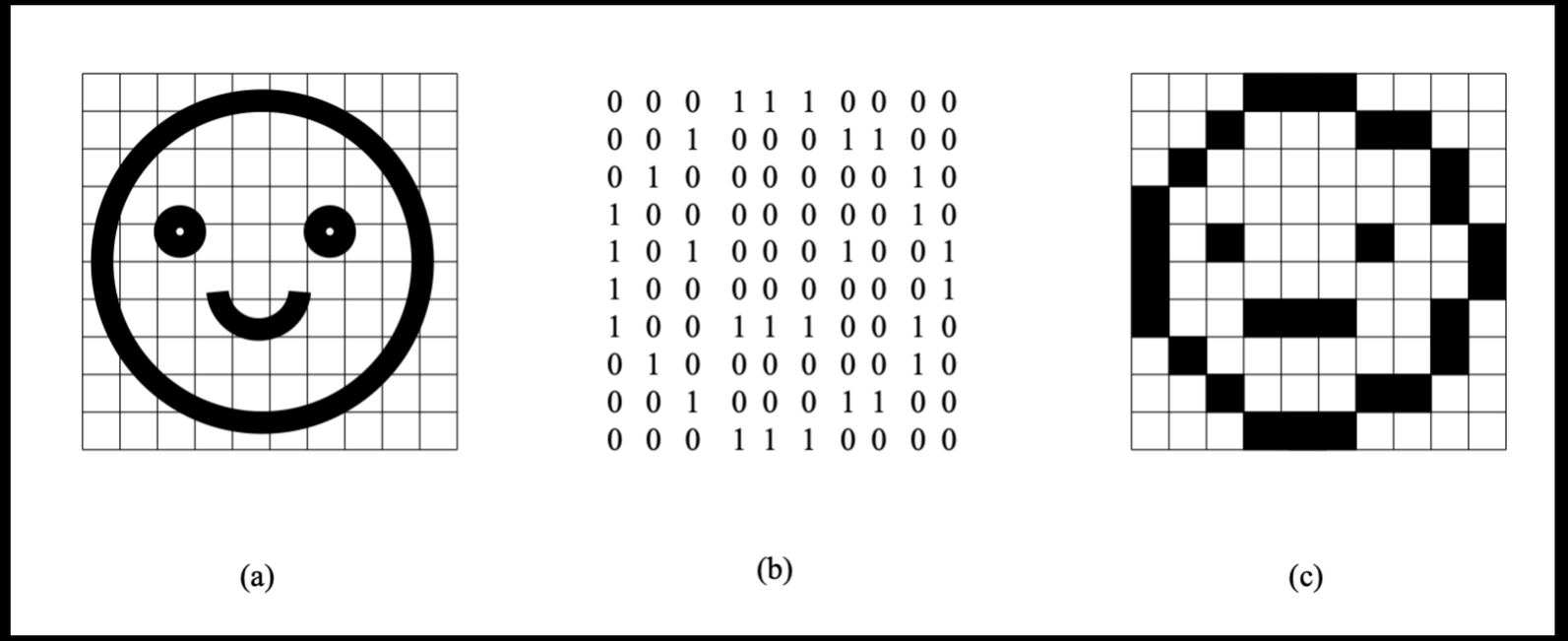


https://sjra.com/can-you-see-a-brain-tumor-on-an-mri-scan/

Representing a black & white picture using numbers

- Suppose picture is 10cm x 10cm.
- Break it up into 0.1 mm x 0.1 mm squares
- 1000 x 1000 squares. 1 million “pixel”s
- If square is mostly white, represent it as 0.
- If square is mostly black, represent it as 1.
- Picture = 1 million numbers!

Picture, representation, reconstruction



Better representation

- Better representation if picture divided into more cells.
- Pictures with different “gray levels”:
  - use numbers 0, 0.1, …, 1.0 to represent level of darkness rather than just 0, 1.
- Pictures with colours: picture = 3 sequences
  - sequence for red component,
  - sequence for blue component,
  - sequence for green component
- Add up the colours to get the actual colour.

An image processing problem

- Input: sequence P of 1 million numbers (0 or 1)
  - Representing a 10cm x 10cm black and white picture,
  - Given in left to right, top to bottom order.
- A very simple image processing problem:
  - Is there a vertical line in the picture?
- Expressing the problem mathematically: What property does the sequence need to have if it is to contain a vertical line?
  - All 0s, except for 1s in consecutive rows of some column
  - Going down a column = move 1000 positions in the sequence
- 1s in positions i, i+1000, i+2000, i+3000, i+4000,… for some i
- ”Is there a vertical line?” = “Does sequence P satisfy above property?”
- One way of solving the problem:
- Try all values of i..

Does the MRI scan contain a tumour?

- In principle, same as asking whether the picture contains a single vertical line:
- Identify a set of properties that the sequence of numbers representing the picture must satisfy if the picture contains a tumor.
- Identify computations that can check if the given number sequence satisfies the required properties.
- In practice requires enormous ingenuity
- Main concern of the deep subject “Computer Vision/deep learning”

Language/text using numbers

- Define a code for representing letters.
- Commonly used code: ASCII
  (American Standard Code for Information Interchange)
- Unicode for characters of other languages.
- Letter ‘a’ = 97 in ASCII, ‘b’ = 98, …
- Uppercase letters, symbols, digits also have codes. Code also for space character.
- Words = sequences of ASCII codes of letters in the word.
- ‘computer’ = 99, 111,109,112,117,116,101,114.
- Does the word “computer” occur in a paragraph?
- Does a certain sequence of numbers occur inside another sequence of numbers?

Exercise

- Suppose you are given a sentence in a language you cannot understand. Would you still be able to count the number of words in the sentence? Can you express this as a question on sequences of numbers representing the ASCII codes of different characters?
- How will you represent Chess playing as a question on numbers? Start by representing a chess board with pieces on it using numbers.

How numbers are written in a computer?

- A computer has a large set of capacitors
  - 8 GB RAM means $2^{36} \approx 6 \times 10^{10}$ capacitors
  - Each capacitor represents a bit (0 or 1)
    - Low charge represents 0
    - High charge represents 1
- Once you represent 0 and 1, you can represent anything.
- Binary number system
- Electrical circuits are designed to perform basic operations like addition, multiplication, copying.

Binary number system

- A sequence $a_n\, a_{n-1} \ldots \;\; a_1\, a_0\, a_{-1} \ldots a_{-k}$ of 0s and 1s
- Will represent the number
  $a_n\, 2^n + a_{n-1}\, 2^{n-1} + \ldots + a_1\, 2^1 + a_0\, 2^0 + a_{-1}\, 2^{-1} + \ldots + a_{-k}\, 2^{-k}$
- Converting decimal integer v to binary
  - Divide v by 2, remainder gives a_0
  - Repeat previous step with the quotient to get a_1, a_2, …
- Converting fraction f to binary
  - If $f > 0.5$, $a_{-1} = 1$ and 0 otherwise.
  - Similarly other bits…

Positive/negative numbers

- Typically number of capacitors used for an integer
  - 8, 16, 32, 64
- One of the bits (leftmost) indicates sign
- -25 will be represented as
  - 10000000000000000000000011001
- Numbers stored from -2^{31} from $2^{31}-1$
- Two's complement